UNIVERSITY INSTITUTE OF ENGINEERING

Department of Computer Science & Engineering

(CSE-3rd Sem)

Subject Name: ADS

Subject Code: 25CSH-202

Submitted to:
Faculty name: MD Tauseef

Submitted by:
Namanjot Kaur
25BCS11711
Krishna Kumari
25BCS11890
Gunjan Sharma
25BCS12387

CHANDIGARH UNIVERSITY
Discover. Learn. Empower.

INDEX

Sr no	Title	Page
1	Topic & Project Abstract	3
2	Objectives & Problem Statement	3
3	System Architecture & Codebase Breakdown	4
4	Applied Data Structures and Algorithms (DSA)	6
5	Mathematical & Statistical Modeling Pipeline	8
6	Implementation Explanation & Runtime Flow	8
7	Performance Evaluation & Edge-Case Analysis	9
8	Future Scope & Scalability Roadmap	10
9	Appendix & Research References	10

1. TOPIC & PROJECT ABSTRACT

Project Title: Gatsby Graph: A Multi-Adaptation Literary-to-Cinema Divergence Analytics Engine and Interactive Web Platform.

Gatsby Graph is a comparative media analytics platform and client-side ingestion engine built to answer a central question: when a novel is adapted into a film or television miniseries, which version achieves higher standing within its own medium, and by what margin? Directly subtracting raw ratings across platforms is statistically flawed. Literary databases like OpenLibrary and Goodreads use a 5-star scale where ratings cluster tightly between 3.7 and 4.3, whereas film databases like IMDb and TMDB use a 10-point scale with wider variance between 5.0 and 8.5. Furthermore, obscure editions suffer from small-sample rating volatility, and classic novels—such as Dune, Wuthering Heights, Pride and Prejudice, Rebecca, and Little Women—have been adapted multiple times across different decades, turning a simple comparison into a one-to-many graph representation problem.

To solve this without relying on heavy third-party frontend frameworks, Gatsby Graph uses an algorithmic pipeline that smooths raw ratings via empirical Bayesian shrinkage, converts scores into within-medium percentiles (0th to 100th), and calculates a signed Adaptation Divergence Score. The project runs in two modes: a local server mode backed by Java and relational storage, and a static cloud deployment on GitHub Pages powered by a browser-side ingestion engine. This report explains the architecture of the codebase and focuses on the applied Data Structures and Algorithms (DSA)—including hash tables, adjacency lists, bitwise DJB2 string hashing, multi-token substring search, custom comparator sorting, N-ary DOM tree traversal, and promise-raced asynchronous state machines—that allow the system to ingest, deduplicate, and render multi-era adaptations in real time.

2. OBJECTIVES & PROBLEM STATEMENT

2.1 Problem Statement

Building a reliable book-versus-film comparison engine requires solving four core technical challenges:

- **Incompatible Rating Scales:** Book ratings sit on a [1, 5] scale (median near 3.92), while movie ratings sit on a [1, 10] scale (median near 6.50). Simple multiplication distorts the comparison because each community votes with different variance and skew.
- **One-to-Many Adaptation Cardinality:** A single novel often has multiple screen

adaptations across decades (e.g., Dune in 1984, 2021, and 2024; Wuthering Heights in 1939, 1992, 2009, and 2011). Storing flat records by book title overwrites older films, while ungrouped rows duplicate book metadata.

- **Browser CORS Limits & Unreliable External APIs:** Live ingestion queries OpenLibrary, Wikipedia's REST API, and IMDb's suggestion buckets. In a browser environment, Cross-Origin Resource Sharing (CORS) blocks, regional catalog restrictions, and slow proxy servers can easily freeze the user interface.
- **Visual Desynchronization:** Loading high-resolution book covers and film posters from separate Content Delivery Networks (CDNs) creates race conditions. If one image finishes downloading 600ms before the other during home page transitions, the layout stutters.

2.2 Engineering Objectives

- **Model Multi-Adaptation Graph Relationships:** Store canonical novels as parent nodes linked to an adjacency list of film adaptations, expanding them into composite keys (bookSlug + '-' + filmYear) at runtime.
- **Generate Deterministic Link Identifiers:** Apply a bitwise DJB2 rolling hash function so every dynamically generated adaptation receives a permanent, reproducible numeric ID across page reloads and shared links.
- **Deliver Instant Search and Custom Sorting:** Support sub-millisecond multi-token search across titles, authors, directors, and years, paired with custom comparator sorting for point spread, medium lead, and release year.
- **Prevent Network Hangs via Promise Racing:** Wrap all external API calls in state-locked timeout timers so slow endpoints fail gracefully within 2.8 seconds, caching resolved adaptations in sessionStorage.
- **Synchronize Image Decoding in RAM:** Preload and decompress both book cover and movie poster pixel buffers off-screen using Image.decode() before triggering UI transitions.

3. SYSTEM ARCHITECTURE & CODEBASE BREAKDOWN

The codebase is organized into four clean layers: (1) the Data & Ingestion Engine, (2) the UI Controller & DOM Engine, (3) the Semantic HTML View Templates, and (4) the CSS Design System & Local Java Server. To ensure identical routing on both localhost:8080 and GitHub Pages (/gatsby_graph/), the files in web/ are mirrored to the repository root alongside a .nojekyll marker.

File / Module	**System Layer**	**Core Responsibility in the Project**
web/data.js (and root data.js)	Data & Ingestion Engine	Stores the multi-adaptation registry, computes Bayesian shrinkage and DJB2 hashes, runs timeout-guarded API fetches, and manages sessionStorage.
web/app.js (and root app.js)	UI Controller & DOM Engine	Detects the active page, queries DOM nodes, runs multi-token search and comparator sorting, handles Sync clicks, and preloads showcase images.
index.html	View Template (Home)	Defines the split hero section, 3D floating card showcase, aggregate statistics strip, and featured comparisons list.
cases.html	View Template (Directory)	Contains the live search input, category filter chips, sort selector, dynamic Sync button trigger, and row list container.
case.html	View Template (Case Detail)	Renders the anti-flicker head script, side-by-side book and film frames, percentile progress bars, and sibling adaptation links.

rankings.html & method.html	View Templates (Analysis)	Displays sorted leaderboards (book lead, film lead, compression ratio) and documents the statistical scoring pipeline.
web/styles.css (and root styles.css)	Styling & Motion Layer	Defines CSS custom variables, Flexbox and CSS Grid layouts, responsive breakpoints, cursor spotlight gradients, and 3D keyframe animations.
src/ (Main.java, WebServer.java)	Local Backend Server	Serves static files on port 8080, exposes local REST endpoints (/api/cases, /api/sync), and interfaces with local relational storage.

3.1 Data and Ingestion Engine (data.js)

Wrapped inside an Immediately Invoked Function Expression (IIFE) under strict mode, data.js keeps all internal helper functions private and exposes only window.GG. It holds KNOWN_ADAPTATIONS, an in-memory hash table mapping normalized book slugs to a parent novel record and an array of film adaptations. On startup, load() checks window.location.hostname: on localhost, it queries /api/cases with an 800ms timeout; on GitHub Pages (*.github.io), it skips the local server check in 0ms to avoid 404 console errors.

When a user searches for a book that is not in the catalog and clicks Sync, window.GG.syncTitle(query) fires parallel requests to OpenLibrary (for publication year, author, page count, cover ID, and rating), Wikipedia's REST Search and Page Summary APIs (for fair-use film posters, release years, and director names), and raced CORS proxies for IMDb's suggestion API. Discovered films are deduplicated by release year, scored against the book, saved in sessionStorage, and merged into the active catalog.

3.2 UI Controller and DOM Manipulator (app.js)

All five HTML pages share app.js. On load, boot() fetches the dataset from window.GG.load() and checks which container node exists in the current DOM tree (.hero, [data-cases-root], .casehero, or [data-rankings-root]) to run the matching initializer.

On cases.html, initCases() attaches listeners to the search box, filter chips, and sort dropdown. Every change triggers draw(), which filters the array in memory, sorts the matches, escapes special HTML characters to prevent XSS, and writes the resulting cards into the list container. When a query returns zero matches, draw() injects the #live-sync-btn button protected by an isSyncing boolean lock. On case.html, renderCaseDetail() reads the ?id= query parameter from the URL, finds the matching object, renders the book cover and movie poster (verifying that current.poster !== current.cover so a book cover is never duplicated into the film slot), and sets the widths of the orange and cyan percentile bars.

3.3 HTML View Templates & CSS Styling Engine (styles.css)

The five HTML templates use relative links (href='cases.html') so navigation works seamlessly inside GitHub Pages subdirectories. In case.html, a tiny script in <head> hides the page body for a few milliseconds when ?id= is present until app.js swaps in the requested book, preventing any flash of default text. Meanwhile, styles.css uses CSS custom properties (--accent orange for books, --support cyan for films), combines 1D Flexbox bars with 2D CSS Grid layouts, and applies 1100px 3D perspective keyframe animations (floatBook and floatFilm) on the home page showcase.

4. APPLIED DATA STRUCTURES AND ALGORITHMS (DSA)

Every core feature in Gatsby Graph—from instant multi-adaptation lookup to live search, sorting, and network fault tolerance—is built directly on foundational Data Structures and Algorithms.

4.1 Hash Tables for O(1) Lookup and Collision Merging

JavaScript associative objects act as in-memory hash tables in three critical places:

- **Canonical Book Lookup (KNOWN_ADAPTATIONS):** Books are keyed by their normalized slug ('dune', 'wuthering-heights'). Expanding a book into its film adaptations requires a single O(1) average-time hash table lookup instead of an O(N) linear scan.
- **Year-Keyed Adaptation Merging (byYear Map):** When syncing a title, both Wikipedia and IMDb may return metadata for the same film year (e.g., 1996 Trainspotting). Inserting results into a hash map keyed by integer filmYear merges complementary fields (Wikipedia's parsed director + IMDb's high-res poster) in O(1) time without nested O(M^2) loops.
- **Composite-Key Deduplication (existingKeys Set):** When combining starter rows with user-synced sessionStorage rows, a hash set tracks composite keys (bookSlug + '-' + filmYear), guaranteeing zero duplicate rows in O(N) linear time.

4.2 One-to-Many Adjacency List Flattening (Graph Representation)

A novel and its film adaptations form a two-level directed star graph where the central parent node is the book and outgoing edges point to k film adaptation nodes. In KNOWN_ADAPTATIONS, this graph is stored as an adjacency list (one parent book object containing a films[] array). At runtime, convertKnownToRows() traverses each parent's adjacency list and flattens every (Book -> Film_i) edge into an independent CaseRow vertex for sorting and filtering, running in O(B * k) time for B books and k adaptations.

4.3 Bitwise Polynomial Rolling Hash (DJB2) for Deterministic IDs

Dynamically expanded or synced adaptations do not have a database auto-increment ID. Using random numbers would break bookmarked URLs (case.html?id=5242) on page refresh. Instead, data.js hashes the composite slug string S (e.g., 'dune-1984') using Daniel J. Bernstein's bitwise DJB2 recurrence:

Bitwise Hash Recurrence: $h_i = ((h_{i-1} << 5) - h_{i-1}) + ASCII(S[i]) = 31 * h_{i-1} + ASCII(S[i])$

Using a left bit-shift (h << 5) minus h performs multiplication by 31 directly in CPU registers. The final integer is mapped into the ID range via modular arithmetic: linkId = 5000 + (|h| mod 4000) + idx. Because slug length L is under 40 characters, ID generation runs in O(1) constant time and produces the exact same ID on every device.

4.4 String Canonicalization & Multi-Token Substring Search

To let users search flexibly (e.g., 'dune 1984', 'david lynch', or 'pride & prejudice'), search processing happens in two steps:

- **Regex Normalization (toSlug & stripArticles):** Converts text to lowercase, expands '&' to ' and ', standardizes 'grey' to 'gray', collapses punctuation into hyphens, and strips leading articles ('the-', 'a-', 'an-').
- **Multi-Token Set Intersection (matchesQuery):** For each row r of N total cases, matchesQuery builds a lowercase haystack string H(r) combining book title, film title, author, director, and both release years. The user's search query is split by whitespace into k tokens {t_1, ..., t_k}. A row matches if every token t_j is a substring of H(r). This runs in O(N * k * M) time (where M ~ 80 characters), filtering the entire catalog in under 1 millisecond.

4.5 Comparison-Based Sorting with Custom Comparators (Timsort)

To reorder cases on cases.html and rankings.html without altering the master array, app.js slices the filtered array and calls Array.prototype.sort()—which uses Timsort (a hybrid of Merge Sort and Insertion Sort running in O(N log N) worst-case and O(N) best-case time)—with custom comparator functions C(a, b):

- **Biggest Gap (div-desc):** C(a, b) = |b.divergence| - |a.divergence|, placing the largest book-vs-film disagreements first.
- **Directional Lead (book-ahead / film-ahead):** Sorts by signed divergence (b.divergence - a.divergence or a.divergence - b.divergence).
- **Compression Density & Chronology:** Sorts by pages per minute (b.pagesPerMinute - a.pagesPerMinute) or film release year (b.filmYear - a.filmYear).

4.6 N-ary DOM Tree Traversal & Promise-Raced State Machines

- **DFS Tree Traversal & Batch Rendering:** The HTML DOM is an N-ary tree. Helpers $(sel) and $$(sel) run Depth-First Search (DFS) to locate target nodes. Rather than inserting rows one by one (which triggers O(N) browser layout recalculations), draw() joins all row

HTML strings in memory and updates innerHTML in a single O(1) DOM tree swap.

- **Promise.race Timeout Guards & Double-Buffering:** In data.js, withTimeout() races every network fetch against a setTimeout timer using a boolean lock (done = false), guaranteeing that slow APIs resolve a clean fallback within 2.8 seconds. On the home page, preloadImage() decodes both book and movie images in off-screen RAM via Image.decode() before swapping DOM sources.

DSA Concept	**Codebase Location**	**Time Complexity**	**Space Complexity**	**Purpose in Gatsby Graph**
Hash Table / Map	data.js (KNOWN_ADAPTATIONS, byYear)	O(1) avg lookup	O(B + K) nodes	Instant catalog lookup and year-based film deduplication.
Adjacency List Flattening	data.js (convertKnownToRows)	O(B * k) traversal	O(N) CaseRows	Expands 1 parent novel into k era-specific film adaptation rows.
DJB2 Bitwise Rolling Hash	data.js (slug -> linkId)	O(L) ~ O(1)	O(1) auxiliary	Produces deterministic numeric IDs for shareable case URLs.
Multi-Token Substring Search	app.js (matchesQuery)	O(N * k * M)	O(k) tokens	Live filtering across titles, authors, directors, and years.
Timsort Comparator Sorting	app.js (initCases, initRankings)	O(N log N)	O(N) subarray	Reorders directory and leaderboards by gap, lead,

				or year.
N-ary Tree DFS & Batch Swap	app.js ($, $$, draw)	O(V) / O(N)	O(N) string	Finds DOM hooks and renders all cards in a single reflow.
Promise.race / Mutex FSM	data.js (withTimeout), app.js	Bounded <= 2.8s	O(P) promises	Prevents API hangs and locks concurrent Sync button clicks.

5. MATHEMATICAL & STATISTICAL MODELING PIPELINE

To compare book ratings and movie ratings fairly, Gatsby Graph processes raw community scores through a four-stage statistical pipeline:

5.1 Empirical Bayesian Rating Shrinkage

A book with a 4.8 average from only 10 voters is statistically less reliable than a classic with 200,000 voters. To prevent small-sample outliers from skewing the rankings, raw averages are pulled toward the medium's global mean using Bayesian shrinkage:

Bayesian Shrinkage: $ShrunkScore = [(v + 15) / (v + 65)] * R + [50 / (v + 65)] * C$

Here, R is the book's raw average rating, v is its vote count, C = 3.92 is the global prior mean across OpenLibrary books, and m = 50 is the smoothing weight. When vote count v is large, the formula trusts R; when v is small, the score shrinks safely toward 3.92.

5.2 Percentile Normalization, Divergence Score & Compression Density

- **Within-Medium Percentiles (P_book and P_film):** Shrunk ratings for both the novel and the film are mapped onto a [0.00, 1.00] percentile rank within their own medium, eliminating the mismatch between 5-star and 10-point scales.
- **Signed Divergence & Dead-Zone Threshold:** The point spread is computed as D = round((P_book - P_film) * 100). A +/- 2 point dead zone classifies D > +2 as BOOK_OUTRANKS_FILM ('book ahead'), D < -2 as FILM_OUTRANKS_BOOK ('film ahead'), and [-2, +2] as ALIGNED ('dead heat').
- **Adaptation Delay & Pages Per Minute (PPM):** Adaptation delay is Delay = max(0, Year_film - Year_book). Narrative compression density is computed as PPM = round((PageCount * 100) / RuntimeMinutes) / 100, measuring how many pages of prose are compressed into each minute of film.

6. IMPLEMENTATION EXPLANATION & RUNTIME FLOW

6.1 Cold Boot & Multi-Adaptation Expansion

When the site loads, boot() calls window.GG.load(). The engine reads any previously synced cases from sessionStorage and runs expandBackendRows(). Using O(1) hash lookups against KNOWN_ADAPTATIONS, single book entries are expanded into all of their historical film adaptations—for instance, expanding Frank Herbert's Dune (1965) into Denis Villeneuve's Dune:

Part One (2021), Dune: Part Two (2024), and David Lynch's Dune (1984), each with its own deterministic DJB2 linkId, ISBN cover, and theatrical poster.

6.2 Live User Search & Dynamic Ingestion ('Sync' Workflow)

When a user searches cases.html for a title not yet in the catalog (such as 'trainspotting' or 'fear and loathing in las vegas'), matchesQuery() returns zero rows and draw() renders the Sync button (#live-sync-btn). Clicking Sync locks the button (isSyncing = true) and runs buildDirectCasesForTitle() across three parallel API streams:

- **1. OpenLibrary Bibliographic Stream:** Fetches the novel's first publication year, author name, page count, community rating, and cover ID in ~300ms.
- **2. Wikipedia REST Summary Stream:** Searches Wikipedia for matching film articles and queries /api/rest_v1/page/summary/<Title>. This endpoint returns fair-use theatrical movie posters alongside the opening summary paragraph, from which regex patterns extract the exact film release year and director name.
- **3. Raced IMDb Suggestion Stream:** Concurrently races CORS proxies against IMDb's suggestion CDN (v3.sg.media-imdb.com) to capture additional adaptation years and high-resolution Amazon posters.

All streams are bounded by withTimeout() at 2.8 seconds. The merged adaptations are scored, saved to sessionStorage, and rendered immediately into the directory.

6.3 Case Detail View & Lockstep Hero Showcase

Clicking any row opens case.html?id=<linkId>, where renderCaseDetail() populates the side-by-side book cover and film poster frames, animates the percentile bars, and links sibling adaptations of the same novel. On index.html, startHeroRotator() cycles through featured pairs every 5.5 seconds—waiting for preloadImage() and Image.decode() to finish in RAM before fading between pairs so both images swap at the exact same millisecond.

7. PERFORMANCE EVALUATION & EDGE-CASE ANALYSIS

Testing on live GitHub Pages across Indian and global CDN edges surfaced four critical real-world edge cases that were resolved through targeted engineering fixes:

- **Eliminating Proxy Rate-Limiting & Infinite Sync Hangs:** Early versions queried public CORS proxies on page load for all starter books, triggering HTTP 429 rate limits when a user clicked Sync. Baking verified CDN poster URLs into KNOWN_ADAPTATIONS reduced startup proxy calls to zero, while replacing AbortController chains with pure Promise.race timers (withTimeout) guaranteed that Sync never hangs.
- **Bypassing Regional iTunes Empty Responses & 1x1 Blank Pixels:** Network diagnostics showed that Apple's iTunes Movie API returns zero results for Indian IP addresses, while OpenLibrary's /b/title/ cover URL returns a 1x1 blank pixel instead of a 404 error when a cover is missing. We removed /b/title/ fallbacks completely and switched film discovery to Wikipedia's globally accessible REST Page Summary API.
- **Preventing Hotlink Blocks (referrerpolicy='no-referrer'):** Amazon CloudFront and Wikimedia image servers occasionally block requests carrying third-party Referer headers. Adding referrerpolicy='no-referrer' to all <img> tags and off-screen Image preloaders ensured 100% image delivery.
- **Preventing Double-Cover Duplication:** When a film lacked a poster in earlier builds, data.js fell back to copying the book cover URL into the film poster field, causing two identical book covers to appear side by side. Removing the cover fallback from poster assignment in data.js and enforcing current.poster !== current.cover in app.js guaranteed that only genuine film posters render in the movie frame.

8. FUTURE SCOPE & SCALABILITY ROADMAP

- **1. In-Memory Prefix Trie & Inverted Index:** As the catalog grows beyond 100,000 adaptations, replacing the O(N * k * M) linear scan in matchesQuery() with a Prefix Trie (Digital Search Tree) and Inverted Token Index will reduce search lookup time to O(L_query), independent of dataset size.
- **2. Serverless Edge Persistence (Cloudflare Workers + Distributed SQLite):** Currently, user-synced books are stored per browser tab in sessionStorage. Connecting syncTitle() to a lightweight serverless edge worker backed by a cloud SQLite replica (such as Turso or Cloudflare D1) will permanently save newly synced adaptations for all visitors worldwide.

- **3. Temporal Sentiment & Inflation-Adjusted Box Office Tracking:** Tracking how a film's percentile rank shifts over decades (capturing cult classics like 1999 Fight Club) alongside CPI inflation-adjusted box-office multiples.
- **4. NLP Bipartite Graph Scene Matching:** Using Natural Language Processing embeddings to construct a bipartite graph matching book chapters to film scenes, quantifying plot faithfulness alongside rating divergence.

9. APPENDIX & RESEARCH REFERENCES

9.1 Deployment & Repository Links

- **Live GitHub Pages Deployment:** https://cinematicbug.github.io/gatsby_graph/
- **GitHub Source Repository:** https://github.com/cinematicbug/gatsby_graph
- **Git Branch & Tag Architecture:** v1.0-stable-design (Core Baseline) and main / ui-motion (Multi-Adaptation & 3D Showcase Engine)

9.2 Technical & Academic References

- **1. OpenLibrary Search & Covers API:** Internet Archive Developer Documentation (https://openlibrary.org/developers/api) — used for first-edition book metadata, ISBN-13 covers, and ratings.
- **2. Wikimedia REST v1 Search & Page Summary API:** MediaWiki REST API Specification (https://en.wikipedia.org/api/rest_v1/) — used for CORS-native film discovery, fair-use movie posters, and director/year extraction.
- **3. IMDb Suggestion CloudFront API:** Amazon Media Suggestion Endpoint (v3.sg.media-imdb.com) — used for multi-era theatrical release verification and poster hydration.
- **4. Empirical Bayes Shrinkage:** Efron, B., & Morris, C. (1977). 'Stein's Paradox in Statistics.' Scientific American, 236(5), 119-127 — statistical basis for shrinking low-vote ratings toward the global medium mean.
- **5. Data Structures & Algorithms:** Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). Introduction to Algorithms (4th ed.). MIT Press — reference for hash tables, string hashing, N-ary trees, and sorting complexity.